

For this assignment, I created a Personal Finance Tracker using Python. The goal was to create a simple, nonetheless, practical tool to help users manage their income and expenses in an effective way. I designed the system with key programming principles, including object-oriented programming (OOP), clean code practices based on PEP 8 (van Rossum, Warsaw and Coghlan, 2001), and a focus on test-driven development (TDD) (Beck, 2002).

From the start, I needed the project to be not only functional but also flexible. Users could add, edit, and remove transactions, filter them by date or category, set spending limits for different areas, and get a clear overview of their finances — including total income, expenses, and current balance. To make the code easier to manage, I split the program into three main classes: Transaction, Budget, and FinanceTracker. Each one handles a specific part of the process, keeping the logic clean and well organised (Gamma et al., 1994).

I decided to build this tool as a command-line interface (CLI), as it allowed me to focus on the structure and logic without the need of a graphical interface. The Python script runs in the terminal and interacts with users through straightforward prompts and menus.

When building the application, I followed the SOLID principles, mainly in the idea that each class should do only one thing. These principles help create software that is straightforward to maintain, scale, and extend (Martin, 2002). This helped me write code that was easier to read and improve over time. I also tried to apply an Agile way,

adding features gradually and testing them step by step rather than trying to build everything in one go.

In the early stages, I focused on getting the basic features working — like adding and viewing transactions. Along the project, I used the PEP 8 style guide to keep everything consistent. I paid attention to naming conventions, indentation, line length, and comments, which made the code more readable and helped prevent mistakes. For example, class names like `FinanceTracker` follow the CamelCase style, while functions like `add_transaction` use snake_case.

To make sure everything worked correctly, I used the pytest framework for unit testing. Writing tests helped me confirm that each component worked as expected and allowed me to catch bugs early in the development process (Krekel, Oliveira and Pfannschmidt, 2004). It also made sure that the code remained stable as I introduced new features.

By saying this, there's still room for improvement. Currently, all transactions are saved in memory — so if the program closes, the data disappears. In the future, I would like to add persistent storage using a database like SQLite or file formats like JSON or CSV. I am also considering creating a graphical interface using Tkinter or building a web version with a framework like Streamlit.

To sum up, this project gave me the chance to apply important programming concepts in a practical way. It reflects what I have learnt about structuring code, following development standards, and thinking critically about how users interact with software.

References:

Beck, K. (2002) *Test-driven development: By example*. Boston: Addison-Wesley.

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) *Design patterns: Elements of reusable object-oriented software*. Boston: Addison-Wesley.

Krekel, H., Oliveira, B. and Pfannschmidt, R. (2004) *pytest documentation*. Available at: <https://docs.pytest.org/en/stable/> (Accessed: 26 March 2025).

Martin, R.C. (2002) *Agile software development: Principles, patterns, and practices*. Upper Saddle River, NJ: Prentice Hall.

van Rossum, G., Warsaw, B. and Coghlan, N. (2001) *PEP 8 – Style Guide for Python Code*. Python Software Foundation. Available at: <https://peps.python.org/pep-0008/> (Accessed: 26 March 2025).